Self-Supervised Contrastive Graph Clustering & Influence Augmented Contrastive (IAC) loss



This repository contains a PyTorch implementation of "SCGC : Self-Supervised Contrastive Graph Clustering".(https://arxiv.org/abs/2204.12656)

The repo, inlcuding data sets and pretrained models are, has been forked initially from SDCN. We use also use the model code from AGCN and portions of contrastive loss code from Graph-MLP.

Setup

- Our code was tested on CUDA 11.3.0, python 3.6.9, pytorch 1.3.1.
- `pip install -q munkres` is needed for the hungarian algorithem, for the evaluation metrics
- The code also run on Google colab (2022 April) with no modifications, with munkres installed.

Note : SCGC is able to run with no GPU if the GPU timing code is commented out.

Datasets

The dataset contains 2 folders, `data` and `graph`. Please obtain them from the dataset Google drive links. You will need to set `--data_path` to the parent folder containing `data` and `graph`. Please note that the `data` folder contains the pre-trained `.pkl` models. We directly use these pre-trained models from SDCN.

Usage

- All parameters are defined in train.py with comments and explanations.
- To run SCGC on the 6 datasets, for 10 iterations, use the following. This code has GPU time and memory profiling enabled, which can be turned off by commenting relevant code. Our published ACC,NMI,ARI and F1 was run with profiling commented.

```
python train.py --name usps --iterations 10 --epochs 200 --model SCGC --ver
python train.py --name hhar --iterations 10 --epochs 200 --model SCGC --ver
python train.py --name reut --iterations 10 --epochs 200 --model SCGC --ver
python train.py --name acm  --iterations 10 --epochs 200 --model SCGC --ver
python train.py --name dblp --iterations 10 --epochs 200 --model SCGC --ver
python train.py --name cite --iterations 10 --epochs 200 --model SCGC --ver
```

- To replicate the SCGC*, which uses Augmented Contrastive (IAC) loss and 50% less model parameters, run the following.

```
python train.py --name usps --iterations 10 --epochs 200 --model SCGC_TRIM
python train.py --name hhar --iterations 10 --epochs 200 --model SCGC_TRIM
python train.py --name reut --iterations 10 --epochs 200 --model SCGC_TRIM
python train.py --name acm  --iterations 10 --epochs 200 --model SCGC_TRIM
python train.py --name dblp --iterations 10 --epochs 200 --model SCGC_TRIM
python train.py --name cite --iterations 10 --epochs 200 --model SCGC_TRIM
```

The usps output is

```
Namespace(alpha=4.0, batch_size=2048, beta=0.1, cuda=True, data_path='/cont
---------------PROFILING CODE--------------
Loaded PAE acc:0.7098  nmi:0.6748  ari:0.5874  f1:0.6968
-------------------------------------------------------  ------------  ----
                                                   Name    Self CPU %
-------------------------------------------------------  ------------  ----
                                       _MODEL_TRAIN_ALL        27.73%
                                           _MODEL_TRAIN         0.06%     1
                                              _MODEL_KL         0.01%
                                            _MODEL_DIST         1.03%
                                     _MODEL_CONTRASTIVE         0.03%
-------------------------------------------------------  ------------  ----
Self CPU time total: 182.898s
```

```
Self CUDA time total: 50.388s
Z:acc-nmi-ari-F1-gpu-clock:  0.8489,  0.8489,  0.8489,  0.0000,|, 0.8411,
Namespace(alpha=1.0, batch_size=2048, beta=10.0, cuda=True, data_path='/con
```

The line `Z:acc-nmi-ari-F1-gpu-clock` gives the min,max,avg,std of ACC,NMI,ARI,F1,GPU,CPU followed by || and all the args.
Profiling `_MODEL_XXX` contexts capture logical model functions and training. Please see the code for more information.

- To reun the prfiling for AGCN and SGCN we used the following. These use the SDCN and AGCN defaults.

```
python train.py --name usps --iterations 1 --epochs 200 --model SDCN --verb
python train.py --name hhar --iterations 1 --epochs 200 --model SDCN --verb
python train.py --name reut --iterations 1 --epochs 200 --model SDCN --verb
python train.py --name acm  --iterations 1 --epochs 200 --model SDCN --verb
python train.py --name dblp --iterations 1 --epochs 200 --model SDCN --verb
python train.py --name cite --iterations 1 --epochs 200 --model SDCN --verb

python train.py --name usps --iterations 1 --epochs 200 --model AGCN --verb
python train.py --name hhar --iterations 1 --epochs 200 --model AGCN --verb
python train.py --name reut --iterations 1 --epochs 200 --model AGCN --verb
python train.py --name acm  --iterations 1 --epochs 200 --model AGCN --verb
python train.py --name dblp --iterations 1 --epochs 200 --model AGCN --verb
python train.py --name cite --iterations 1 --epochs 200 --model AGCN --verb
```

Data sources and code

Datasets and code is forked from SDCN. We use also use the model code from AGCN AGCN and portions of contrastive loss code from Graph-MLP. We acknowledge and thank the authors of these works for sharing their code.

Citation

```
@article{kulatilleke2022scgc,
  title={SCGC : Self-Supervised Contrastive Graph Clustering},
  author={Kulatilleke, Gayan K and Portmann, Marius and Chandra, Shekhar S}
```

Created with GitHub README to PDF Converter • github.com/gayanku/readme-to-pdf

```
  journal={arXiv preprint arXiv:2204.12656},
  year={2022}
}
```